A Deep Reinforcement Learning-Based Service Caching and Computation Offloading Algorithm for Multi-UAV-Enabled MEC Systems

Luyinru Yang, Jun Zheng, *Senior Member, IEEE*, and Yuan Zhang

***Abstract*—This paper investigates the joint service caching and computation offloading optimization problem in a multi-UAV-enabled mobile edge computing (MEC) system. The problem is formulated as a non-linear integer programming problem with an objective to minimize the average service delay of all computing tasks generated by ground users in the system. To solve the formulated problem, the original problem is first decomposed into two coupled subproblems, i.e., service caching and computation offloading, and a two-layer learning framework is then introduced to solve the subproblems. The learning framework consists of an upper layer and a lower layer, which are used for service caching policy learning and computation offloading policy learning, respectively. Based on the learning framework, a deep reinforcement learning (DRL)-based joint optimization algorithm, which consists of a lower-layer algorithm and an upper-layer algorithm, is further proposed to perform service caching and computation offloading. The lower-layer algorithm uses a graph neural network (GNN) and an attention mechanism to extract relevance information on the states of ground users in the system and uses a deep Q-network (DQN) to make UAV assignment decisions based on the extracted information. The upper-layer algorithm uses an actor-critic structure to make optimal service caching decisions. Simulation results demonstrate that the proposed algorithm can significantly reduce the average service delay of all computing tasks in the system compared with benchmark algorithms.**

***Index Terms*—UAV, MEC, deep reinforcement learning, service caching, computation offloading.**

I. Introduction

THE rapid development of mobile communication technologies has promoted the explosive growth of intelligent mobile applications, such as augmented reality, live streaming, autonomous driving and interactive gaming [1]. Most of these applications have high quality-of-service (QoS) requirements and typically require substantial computing resources for data processing. Due to their limited computing capacity, many mobile devices, such as smart phones, laptops, and wearables, are unable to satisfy the QoS requirements of their applications. As a promising solution to this issue, mobile edge computing (MEC) is capable of processing data in the proximity of mobile devices or users with more sufficient computing resources deployed at the edge of a mobile network [2], [3]. Mobile users can offload their delay-sensitive or computation-intensive tasks to edge servers for processing, which can significantly reduce the service delay of a computing task generated by a mobile user. However, in many extreme situations where terrestrial infrastructure is not available, such as emergence situations or remote areas, mobile users may not be able to have access to conventional MEC servers. In such situations, unmanned aerial vehicles (UAVs) have been widely regarded as a promising solution to enhancing service performance for mobile users [4] with the advantages of flexible deployment, high mobility, and low cost [5]. An aerial MEC service platform can be quickly built by deploying one or more UAVs equipped with MEC servers. Moreover, it can significantly extend its service coverage as a UAV can fly to proximity of user devices. Due to the limitations of a single UAV in size, weight and power, multiple UAVs can be deployed to better serve mobile users. This makes computation offloading a critical issue in a multi-UAV-enabled MEC (MUeM) system. On the other hand, different mobile applications often rely on different service programs and corresponding databases for processing their tasks. This necessitates service caching, i.e., caching service-related programs or databases in advance on MEC servers, in the system. In this case, the limited onboard cache capacity of a UAV introduces additional constraints on making computation offloading decisions, tightly coupling service caching and computation offloading. Therefore, it is necessary to investigate the joint service caching and computation offloading problem in the system.

In this paper, we investigate the joint service caching and computation offloading optimization problem in an MUeM system. We formulate the problem and introduce a DRL method to solve the formulated problem. The main contributions of this work can be summarized as follows:

1) The investigated problem is formulated as a non-linear integer programming (NLIP) problem with the objective of minimizing the average service delay of all computing tasks generated by ground users in the system.

Received 2 October 2025; revised 31 January 2026; accepted 23 February 2026. Date of publication 3 March 2026; date of current version 16 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62471130 and Grant 62271134, and in part by the National Key R&D Program of China under Grant 2023 YFB4503302. The associate editor coordinating the review of this article and approving it for publication was Q. Ye. *(Corresponding author: Jun Zheng.)*

Luyinru Yang and Yuan Zhang are with the School of Information Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: lyryang@seu.edu.cn; y.zhang@seu.edu.cn).

Jun Zheng is with the School of Information Science and Engineering, Southeast University, Nanjing 210096, China, and also with Purple Mountain Laboratories, Nanjing 211111, China (e-mail: junzheng@seu.edu.cn).

Digital Object Identifier 10.1109/TCCN.2026.3670152

2332-7731 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

2) To solve the formulated problem, we first decompose the problem into two coupled subproblems, i.e., service caching and computation offloading, and then introduce a two-layer learning framework to solve the subproblems. The framework consists of an upper layer and a lower layer. The upper layer is used to learn an optimal policy for making an optimal decision on service caching and the lower layer is used to learn an optimal policy for making an optimal decision on computation offloading.
3) Based on the learning framework, we further propose a DRL-based joint optimization algorithm to perform service caching and computation offloading, which consists of a lower-layer algorithm and an upper-layer algorithm. The lower-layer algorithm uses a graph neural network (GNN) and an attention mechanism to extract relevance information on the states of ground users in the system and uses a deep Q-network (DQN) to make UAV assignment decisions based on the extracted information. The upper-layer algorithm uses an actor-critic structure to make optimal service caching decisions.
4) It is demonstrated through simulation results that the proposed optimization algorithm can significantly reduce the average service delay of all computing tasks in the system compared with three benchmark algorithms.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system model and formulates the problem considered in this paper. Section IV presents the proposed algorithm. Section V presents simulation results to evaluate the performance of the proposed algorithm. Section VI concludes this paper.

II. Related Work

MUeM systems have recently received much attentions and considerable research work has been conducted to investigate the computation offloading, resource allocation, and UAV trajectory design issues in such systems. Luo et al. [6] investigated a joint user allocation, UAV trajectory and bit allocation optimization problem to minimize the total energy consumption in an MUeM system and proposed a two-layer DRL-based scheme to solve the problem. Liu et al. [7] investigated a joint optimization problem in an MUeM network with the objective to minimize a weighted sum energy consumption, and proposed a multi-agent proximal policy optimization (MAPPO)-based approach with Beta distribution and an attention mechanism to solve the problem. Nguyen et al. [8] investigated an average delay minimization problem in a collaborative multi-UAV network through optimizing offloading decisions and communication resource allocation and took dependency among subtasks into consideration. Pervez et al. [9] studied a joint optimization problem in an MUeM network and proposed a three-layer alternating iterative algorithm to solve the problem. Yu et al. [10] investigated a joint computation offloading and trajectory optimization problem in an MUeM system, which considers cooperation and heterogeneity among UAVs. Kuang et al. [11] formulated a joint task offloading, resource allocation, and UAV deployment problem in an MUeM system aiming to maximize the number of tasks offloaded to UAVs and proposed a two-layer optimization method based on differential evolution and convex optimization to solve the formulated problem. Qin et al. [12] studied a joint task scheduling, resource allocation and UAV trajectory optimization problem for air-ground collaborative MEC and proposed an iterative algorithm to minimize a weighted age of information (AoI) of all terrestrial users.

In the context of service caching in an MEC system, related work can also be found in literature. Bi et al. [13] considered a cache-assisted MEC system and studied a joint computation offloading, service caching and resource allocation optimization problem to minimize the computation delay and energy consumption of the mobile user. Zhang et al. [14] studied a joint service caching, computation offloading and resource allocation optimization problem in a general multi-user multi-task system to minimize the overall computation cost and delay cost of the system. A semidefinite relaxation and alternating optimization based approximate algorithm is proposed to solve the problem. Wang et al. [15] studied a joint service caching, communication and computing resource allocation problem to minimize the average system delay in a MEC-enabled IoT scenario and proposed a multiagent algorithm to solve the problem based on a deep graph convolution reinforcement learning method. Shang et al. [16] investigated both computation offloading and service caching in an end-edge-cloud collaborative system and proposed a DRL-based approach for minimizing a weighted sum of time and energy consumption. However, most of the existing work focus on general cellular scenarios and only several studies consider service caching in an MUeM system. For instance, Zheng et al. [17] studied a joint service caching, task offloading, resource allocation and UAV placement optimization problem, aiming at minimizing the maximum task completion delay among all devices and proposed an iterative algorithm to solve the problem based on block coordinate descent (BCD) and successive convex approximation (SCA) techniques. Yan et al. [18] designed a UAV-assisted service provisioning framework for an IoT system and proposed a UAV-enhanced service caching scheme based on potential game. Zhao et al. [19] studied a joint content caching, service placement and task offloading problem in a UAV-enabled MEC network with an objective to maximize the average quality-of-experience (QoE) of the network. Gibbs sampling-based and matching game-based algorithms are designed to solve the problem. Li et al. [20] proposed a joint DRL-based service caching and computation offloading scheme with 3D UAV deployment to enhance the QoS for intelligent connected vehicles in UAV-assisted vehicular edge computing.

In summary, the joint service caching and computation offloading optimization problem in an MUeM system has not been explored sufficiently and thus need more in-depth studies.

III. System Model and Problem Formulation

This section first describes the system model, and then formulates the joint optimization problem considered in this paper.

A. System Model

We consider an MUeM system with N ground users (GUs) and M UAVs, which are denoted by $\mathcal{G} = \{G_n, n = 1, 2, \ldots, N\}$, and $\mathcal{U} = \{U_m, m = 1, 2, \ldots, M\}$, respectively. The UAVs serve as aerial base stations to provide MEC services for the

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

ground users during a service period. Each of the ground users randomly generates computation-intensive tasks and requests for offloading tasks to the UAVs for processing. Compared with the ground users, each of the UAVs is equipped with an MEC server and has more computation resources for task processing. Moreover, one of the UAVs has additional computation resources and serves as a central controller for the UAVs, which is responsible for making decisions for all tasks generated in the system. When a ground user generates a computing task, it will send a request to the central controller for offloading the task. The central controller will first make decisions on UAV assignment and bandwidth resource allocation for the task. Specifically, the central controller will assign a particular UAV and allocate bandwidth resources for offloading the task. Then, the task will be offloaded to its assigned UAV and be processed.

Assume that each UAV in the system operates on a different spectrum band and the spectrum band of each UAV is divided into a set of orthogonal subchannels, which are allocated to the ground users served by the UAV. Thus, each ground user can communicate with a UAV without interference in an orthogonal frequency-division multiple access (OFDMA) manner. The overall system operates in a time-slotted manner during its service period. Moreover, each UAV hovers at a fixed altitude. The three-dimensional coordinate of U_m is denoted by $\boldsymbol{pos}_m = (x_m, y_m, h_U)$. The coordinate of G_n in time slot t (t = 1, 2, 3,...) is denoted by $\boldsymbol{pos}_n(t) = (x_n(t), y_n(t), 0)$.

1) Caching Model: Assume that a computing task can only be processed using a particular service program with a corresponding database. Thus, different computing tasks may need different service programs for processing. In the operation of the system, all service programs are cached on the UAVs in advance. Each computing task needs to be offloaded to a UAV which caches a corresponding service program in advance; otherwise, the computing task cannot be processed successfully.

Let $\mathcal{L} = \{S_l, l = 1, 2, \ldots, L\}$ denote a set of service programs for processing computing tasks in the system, where S_l denotes the l-th service program. Since each UAV has a limited cache capacity, the total size of the service programs cached on a UAV cannot exceed the cache capacity of the UAV. Thus, each UAV can only selectively cache the service programs. We use a service caching matrix $\boldsymbol{C}^{M\times L} = \{c_m^l | U_m \in \mathcal{U}, S_l \in \mathcal{L}\}$ to denote the service caching states of all the UAVs, where $c_m^l = 1$ indicates that S_l is cached on U_m; otherwise, $c_m^l = 0$. Thus, the constraint of a UAV's cache capacity is given by

$$\sum_{l=1}^{L} c_m^l d_l \le C_U, \forall U_m \in \mathcal{U}, \tag{1}$$

where C_U denotes the cache capacity of a UAV and d_l denotes the cache size of service program S_l. The service caching matrix of all the UAVs is determined before the operation of the system starts.

2) Computation Model: Assume that each ground user generates a computing task with probability p_0 in each time slot. A task generated by G_n in time slot t is characterized by $T_n(t) = (D_n(t), F_n(t), l_n(t))$, where $D_n(t)$ denotes the data size of $T_n(t)$, $F_n(t)$ denotes the number of CPU cycles required to process $T_n(t)$ and $l_n(t)$ denotes the type of the service program required to process $T_n(t)$. Each computing task has no maximum tolerant delay requirement.

In a real MEC system, a request for a service program (e.g., image recognition, video processing, or speech recognition) typically exhibits highly skewed popularity. In another word, it follows a long-tail popularity distribution, in which a small number of services dominate the majority of requests. Thus, we use p_l to denote the probability that a task requires S_l for processing, and assume that the probability p_l follows a Zipf distribution [21], i.e.,

$$p_l = \frac{1/l^\alpha}{\sum_{i=1}^{L} 1/i^\alpha}, l = \{1, 2, \ldots, L\}, \tag{2}$$

where α is a coefficient which represents the degree of skew in content popularity. Let $x_n^m(t)$ denote the assignment indicator of task $T_n(t)$, where $x_n^m(t) = 1$ indicates $T_n(t)$ is assigned to U_m and $x_n^m(t) = 0$ otherwise.

Assume that a UAV processes only one computing task at a time and sequentially processes computing tasks according to a first-in-first-out (FIFO) principle. After a computing task is offloaded to a UAV, it will be cached in a computation queue of the UAV and wait for processing. The computation capability of a UAV is measured by its CPU frequency and is denoted by f_C. Thus, the computation delay of $T_n(t)$ is given by

$$\tau_{n,t}^c = \frac{F_n(t)}{f_C}. \tag{3}$$

Meanwhile, since each computing task can only be processed using a corresponding service program, the UAV assignment for $T_n(t)$ must meet the following constraint:

$$\sum_{m=1}^{M} x_n^m(t) c_m^{l_n(t)} = 1. \tag{4}$$

3) Communication Model: The channel between a UAV and a ground user, called an A2G channel, could be either line-of-sight (LoS) or non-line-of-sight (NLoS) according to the probabilistic LoS channel model [22]. According to [22], the LoS path loss and NLoS path loss between G_n and U_m are respectively given by

$$PL_{LoS}^{n,m}(t) = 20\log\frac{4\pi f_U}{c} + 20\log dis_{n,m}(t) + \eta_{LoS}, \tag{5}$$

$$PL_{NLoS}^{n,m}(t) = 20\log\frac{4\pi f_U}{c} + 20\log dis_{n,m}(t) + \eta_{NLoS}, \tag{6}$$

where η_{LoS} and η_{NLoS} denote the additional attenuation factors in LoS and NLoS conditions, respectively, f_U denotes the carrier frequency of a UAV, and $dis_{n,m}(t)$ denotes the distance between U_m and G_n in time slot t. The probability that the channel is a LoS channel is given by

$$p_{LoS}^{n,m}(t) = \frac{1}{1 + a\exp(-b(\theta_{n,m}(t) - a))}, \tag{7}$$

where a and b are environment-dependent coefficients constants, and $\theta_{n,m}(t) = (180/\pi)\cdot\sin^{-1}(h_U/dis_{n,m}(t))$ is the elevation angle between U_m and G_n in time slot t. Accordingly, we have the mean path loss of the A2G channel, i.e.,

$$PL_{n,m}(t) = p_{LoS}^{n,m}(t)PL_{LoS}^{n,m}(t) + (1 - p_{LoS}^{n,m}(t))PL_{NLoS}^{n,m}(t). \tag{8}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

According to Shannon's theorem, the transmission rate of the channel from G_n to U_m is given by

$$R_n^m(t) = b_n^m(t)B\log_2\left(1+\frac{P_G}{10^{PL_{n,m}(t)/10}N_0 b_n^m(t)B}\right), \quad (9)$$

where P_G is the transmission power of a ground user, N_0 is the power spectral density of the noise on the channel, B denotes the bandwidth of an orthogonal subchannel and $b_n^m(t)$ denotes the number of subchannels allocated to G_n by U_m in time slot t.

Since the bandwidth resources of each UAV are limited, the total amount of bandwidth resources allocated to all the ground users by a UAV cannot exceed its available bandwidth, i.e.,

$$\sum_{n=1}^{N} b_n^m(t)\cdot B \le N_B \cdot B, \forall \mathcal{U}_m \in U, \quad (10)$$

where N_B denotes the number of orthogonal subchannels of a UAV.

B. Problem Formulation

We consider the joint service caching and computation offloading optimization problem in an MUeM system.

Let $\boldsymbol{T} = \{T_n(t)|G_n \in \mathcal{G}\}$ denote a set of computing tasks generated by the ground users in the system, $|\boldsymbol{T}|$ denote the total number of computing tasks in T, $\boldsymbol{C} = \{c_m^l|U_m \in \mathcal{U}, S_l \in \mathcal{L}\}$ denote a service caching decision for all the UAVs, $\boldsymbol{X} = \{x_n^m(t)|G_n \in \mathcal{G}, \mathcal{U}_m \in \mathcal{L}\}$ denote a set of UAV assignment decisions for all computing tasks in T, and $\boldsymbol{B} = \{b_n^m(t)|G_n \in \mathcal{G}, U_m \in \mathcal{U}\}$ denote a set of bandwidth resource allocation decisions for all computing tasks in T, respectively. Define the service delay of a computing task as the duration between the instant the task is generated and the instant the task is completed and let $\tau_{n,t}$ denote the service delay of $T_n(t)$. The objective of the problem is to minimize the average service delay of all computing tasks generated by the ground users in the system by jointly optimizing service caching C, UAV assignment X and bandwidth resource allocation B, subject to the cache capacity and bandwidth resource constraints of the UAVs, i.e.,

P1: objective:

$$\min_{\boldsymbol{C},\boldsymbol{X},\boldsymbol{B}} D = \frac{1}{|\boldsymbol{T}|}\sum_{\boldsymbol{T}}\tau_{n,t} \quad (11a)$$

subject to

$$x_n^m(t) \in \{0,1\}, \forall G_n \in \mathcal{G}, U_m \in \mathcal{U}, \quad (11b)$$

$$\sum_{m=1}^{M} x_n^m(t) \le 1, \forall G_n \in \mathcal{G}, \quad (11c)$$

$$b_n^m(t) \in \{0,1,\ldots,N_B\}, \forall G_n \in \mathcal{G}, U_m \in \mathcal{U}, \quad (11d)$$

$$\sum_{n=1}^{N} b_n^m(t) \le N_B, \forall U_m \in \mathcal{U}, \quad (11e)$$

$$c_m^l \in \{0,1\}, \forall S_l \in \mathcal{L}, U_m \in \mathcal{U}, \quad (11f)$$

$$\sum_{l=1}^{L} c_m^l d_l \le C_m, \forall U_m \in \mathcal{U}, \quad (11g)$$

$$\sum_{m=1}^{M} x_n^m(t) c_m^{l_n(t)} = 1, \forall G_n \in \mathcal{G}, \quad (11h)$$

where constraints (11b) and (11c) ensure that each task can only be assigned one UAV, constraint (11d) specifies the range of $b_n^m(t)$, constraint (11e) ensures that the overall bandwidth allocated to all the ground users by a UAV should not exceed its available bandwidth, constraint (11f) specifies the range of c_m^l, constraint (11g) ensures that the total size of the service programs cached on a UAV should not exceed the cache capacity of the UAV, constraint (11h) ensures that each computing task can only be assigned one UAV caching the service program requested by the task.

The problem **P1** formulated in Eq. (11) is an NLIP problem, which is NP-hard and thus is difficult to find an optimal solution. Since all computing tasks in the system are generated randomly in each time slot, the system needs to make decisions on UAV assignment and bandwidth resource allocation for the tasks generated in each time slot. However, conventional optimization algorithms usually have a high computation complexity and thus are unable to find optimal solutions timely. To deal with this issue, we introduce DRL and propose a DRL-based joint optimization algorithm to solve the formulated problem.

IV. Two-Layer Learning Framework and Joint Optimization Algorithm

This section presents the proposed learning framework and the DRL-based joint optimization algorithm.

A. Overview

The formulated joint optimization problem involves service caching and computation offloading. In the considered MEC system, service caching and computation offloading are performed at different time granularities. Computation offloading needs to be performed in each time slot of a service period, while service caching needs to be performed only once in a service period. Thus, we decompose the problem into two sub-problems: service caching and computation offloading. To solve the problem, we first introduce a two-layer learning framework and then design an optimization algorithm based on the framework to make decisions on service caching and computation offloading.

The two-layer learning framework consists of an upper layer and a lower layer, as shown in Fig. 2. The upper layer is used to learn an optimal policy for making an optimal decision on service caching, i.e., obtaining an optimal service caching matrix. The lower layer is used to learn an optimal policy for making an optimal decision on computation offloading, i.e., making optimal decisions on UAV assignment and bandwidth resource allocation. For the two layers, different neural networks are designed for learning optimal policies. The parameters of the neural networks are updated to obtain optimal policies in the learning processes at the two layers.

By using the above learning framework, a computation offloading decision at the lower layer can be optimized with a given service caching matrix determined by the upper layer and a service caching decision at the upper layer can be optimized with a given computation offloading policy given by the lower layer. However, the learning process of the lower layer and that of the upper layer are not completely independent of

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

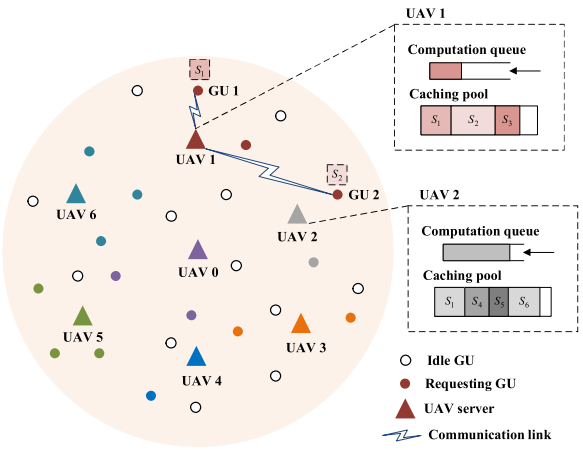


Fig. 1. System model.

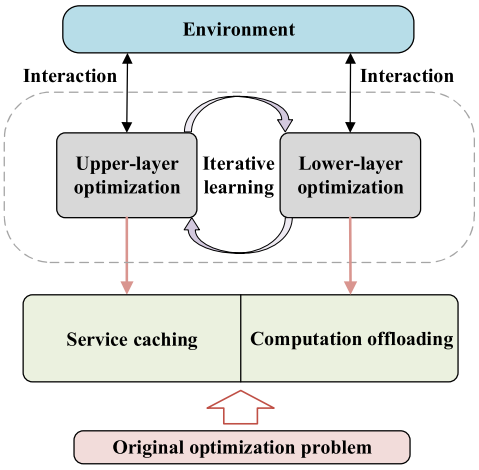


Fig. 2. Illustration of the two-layer learning framework.

each other. A non-converging lower-layer policy will increase the learning difficulty of the upper layer. Thus, the upper layer and the lower layer need to perform iterative learning periodically until convergence.

Based on the two-layer learning framework, the proposed DRL-based joint optimization algorithm consists of an upper-layer optimization algorithm and a lower-layer optimization algorithm, which correspond to the two layers in the framework, respectively. Next we will describe the two algorithms in more details, respectively.

B. Lower-Layer Optimization Algorithm

Given the service caching matrix determined by the upper layer, the objective of the lower-layer optimization is to learn a computation offloading policy to make optimal decisions on UAV assignment and bandwidth resource allocation for each computing task generated in the system such that the average service delay of all computing tasks generated in the system is minimized, i.e.,

P2: objective:

$$\min_{\boldsymbol{X},\boldsymbol{B}} D = \frac{1}{|\boldsymbol{T}|}\sum_{\boldsymbol{T}} \tau_{n,t} \tag{12a}$$

$$\text{subject to} \quad x_n^m(t) \in \{0,1\}, \forall G_n \in \mathcal{G}, U_m \in \mathcal{U}, \tag{12b}$$

$$\sum_{m=1}^{M} x_n^m(t) \le 1, \forall G_n \in \mathcal{G}, \tag{12c}$$

$$b_n^m(t) \in \{0,1,\ldots,N_B\}, \forall G_n \in \mathcal{G}, U_m \in \mathcal{U}, \tag{12d}$$

$$\sum_{n=1}^{N} b_n^m(t) \le N_B, \forall U_m \in \mathcal{U}, \tag{12e}$$

$$\sum_{m=1}^{M} x_n^m(t) c_m^{l_n(t)} = 1, \forall G_n \in \mathcal{G}, \tag{12f}$$

where constraints (12b)-(12f) are the same as constraints (11b)-(11e) and constraint (11h).

To solve **P2**, we first model the problem as a Markov Decision Process (MDP), and then design a lower-layer network and a corresponding optimization algorithm.

In the MDP, the central controller in the system is modeled as an agent, while each ground user in the system is considered as a virtual agent with all its actions decided by the agent. In this way, the lower layer can use a centralized multi-agent reinforcement learning (MARL) method to learn a centralized decision policy. In each time slot, the agent interacts with the environment by obtaining the state of the environment, choosing and executing an action, and obtaining a reward and the new state of the environment in the next time slot. The state, action, reward and new state in each time slot are stored as an experience in a replay buffer. Each time the agent trains its neural networks, a batch of experiences will be sampled from the replay buffer to update the parameters of the networks.

The state, action and reward of the modeled MDP for **P2** are defined as follows:

(1) *State:* The state of the environment at the lower layer includes the states of all UAVs and the local states of all ground users in the system. Let $\boldsymbol{s}(t)$ denote the state of the environment at the lower layer in time slot t, $\{s_m(t)\}$ denote the states of all the UAVs in time slot t, where $s_m(t)$ denotes the state of U_m, including U_m's position and available bandwidth resources, the service caching matrix and the total number of CPU cycles needed for processing the tasks in U_m's computation queue. For each virtual agent, its local state includes its position and the information on its generated computing task.

(2) *Action:* An action that the agent takes in a time slot is to perform UAV assignment and bandwidth resource allocation for all computing tasks waiting for offloading in the time slot. The action taken by the agent in time slot t is denoted by $\boldsymbol{a}(t) = \{\boldsymbol{a}_n(t) | G_n \in \mathcal{G}\}$, where $\boldsymbol{a}_n(t) = \{x_n^m(t), b_n^m(t) | U_m \in \mathcal{U}\}$ denotes the action taken for G_n.

(3) *Reward:* The immediate reward of the system in a time slot is defined as an opposite value of the current average service delay of all computing tasks generated in the system, i.e.,

$$r_t = -D(t), \tag{13}$$

where $D(t)$ denotes the average service delay of all computing tasks which have been generated in the system until time slot t.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

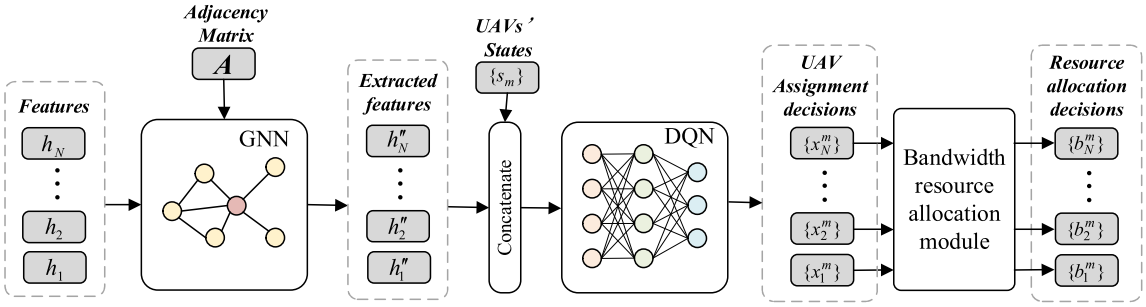


Fig. 3. Illustration of the lower layer.

The lower layer is composed of a GNN module, a DQN module and a bandwidth resource allocation (BRA) module, as illustrated in Fig. 3. The GNN module is used to extract relevance information on the state of each ground user in the system. The DQN is used to choose actions on UAV assignment based on the information extracted by the GNN module. The BRA module is used to make decisions on bandwidth allocation based on the actions chosen on UAV assignment.

1) GNN Module: The GNN module uses a GNN to extract state-relevance information from the features of all ground users in the system and aggregate the extracted information into higher-level features. To use a GNN, we first model all ground users in the system and their local states as an undirected graph $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E})$, where $\boldsymbol{V} = \{v_i | i = 1, 2, \ldots, N\}$ is a set of vertexes representing the ground users and $\boldsymbol{E} = \{e_{ij} | v_i, v_j \in \boldsymbol{V}\}$ is a set of edges representing temporary "neighboring" relations between different ground users in the system. Each vertex in G corresponds to a ground user in the system and the local state of a ground user is the feature of the corresponding vertex in G.

We use an adjacency matrix $\boldsymbol{A}(t) \in \mathbb{R}^{N \times N}$ to represent the temporary "neighboring" relations between the ground users in time slot t. The definition of the adjacency matrix is given by

$$\boldsymbol{A}(t) = \begin{bmatrix} e_{11}(t) & \cdots & \cdots & e_{1N}(t) \\ \vdots & \ddots & & \vdots \\ & & e_{ij}(t) & \\ \vdots & & \ddots & \vdots \\ e_{N1}(t) & \cdots & \cdots & e_{NN}(t) \end{bmatrix} \tag{14}$$

where $e_{ij}(t)$ denotes a temporary neighboring relation between v_i and v_j in time slot t. When both v_i and v_j have a task requesting for offloading in time slot t, $e_{ij}(t) = 1$ and v_j is considered as a neighboring vertex of v_i in time slot t; otherwise, $e_{ij}(t) = 0$. Note that v_i is also viewed as a neighboring vertex of itself here when v_i has a task requesting for offloading.

The features of all the vertexes (i.e., the local states of all the ground users) are input into the GNN together with the adjacency matrix. The GNN will extract state-relevance information from the features and aggregate the extracted information into higher-level features. To improve the accuracy of the extraction and aggregation, the GNN uses an attention mechanism to fit the influences between different vertexes and uses different attention weights to indicate the influences.

Inspired by [23], we use two attention layers to construct a GNN. The first layer is used to extract state-relevance information from the features of all the vertexes and generates a new feature of each vertex. The second layer is used to further extract information from the new feature generated by the first and generates a higher-level feature of each vertex, called output feature of a vertex. The output features of all the vertexes constitute the output of the GNN module.

The first layer is a multi-head graph attentional layer with each head representing a type of influences. The aggregation process at this layer can be expressed as

$$h_i'(t) = \mathop{\Big\Vert}_{z=1}^{Z} \sigma\left(\sum_{j \in \boldsymbol{V}_i} \alpha_{ij}^z(t) \boldsymbol{W}^z h_j(t) \right), \tag{15}$$

where $\|$ denotes a concatenation operation, $h_j(t)$ denotes the feature of vertex v_j in time slot t, $h_i'(t)$ denotes the new feature of v_i in time slot t, Z denotes the number of attention heads on this layer, $\boldsymbol{V}_i$ denotes a set of neighboring vertexes of v_i, $\alpha_{ij}^z(t)$ denotes the attention weight from v_j to v_i for the z-th attention head in time slot t, $\boldsymbol{W}^z$ is a transformation matrix for the z-th attention head, and $\sigma(\cdot)$ is a nonlinearity function (such as a ELU activation function). The attention weight from v_j to v_i in time slot t indicates the influence of v_j's features on v_i. For attention head z, the attention weight is generated as

$$\alpha_{ij}^z(t) = \frac{\exp(\text{LeakyReLU}((\boldsymbol{\beta}^z)^T [\boldsymbol{W}^z h_i(t) \| \boldsymbol{W}^z h_j(t)]))}{\sum_{k \in \boldsymbol{V}_i} \exp(\text{LeakyReLU}((\boldsymbol{\beta}^z)^T [\boldsymbol{W}^z h_i(t) \| \boldsymbol{W}^z h_k(t)]))}, \tag{16}$$

where $(\cdot)^T$ denotes a transposition operation and β^z denotes a weight vector for the z-th attention head.

At the second attention layer, the aggregation process can be expressed by

$$h_i''(t) = \sigma\left(\sum_{j \in \boldsymbol{V}_i} \alpha_{ij}'(t) \boldsymbol{W}' h_j'(t) \right), \tag{17}$$

where $h_i''(t)$ denotes the output feature of v_i in time slot t, $\alpha_{ij}'(t)$ denotes the attention weight of v_j to v_i in time slot t, and $\boldsymbol{W}'$ denotes a transformation matrix at the second attention layer. The attention weights of the second attention layer are generated as

$$\alpha_{ij}'(t) = \frac{\exp(\text{LeakyReLU}(\boldsymbol{\beta}'^T [\boldsymbol{W}' h_i'(t) \| \boldsymbol{W}' h_j'(t)]))}{\sum_{k \in \boldsymbol{V}_i} \exp(\text{LeakyReLU}(\boldsymbol{\beta}'^T [\boldsymbol{W}' h_i'(t) \| \boldsymbol{W}' h_k'(t)]))}, \tag{18}$$

where β' denotes a weight vector of the second attention layer.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

2) DQN Module: The DQN module uses a deep neural network (DNN) to choose actions on UAV assignment based on the output features of all ground users in the system, i.e., the output of the GNN module. The output features of all the ground users are input into the DQN module with the states of all UAVs in the system, i.e., $\{s_m(t)\}$. The DQN module maps the input into a set of action values, and chooses the action with the highest action value for G_n. The DQN module outputs the chosen actions as the decisions for UAV assignment. Note that the action chosen for a ground user is ignored and masked when the ground user has no task requesting for offloading.

3) BRA Module: The BRA module allocates available bandwidth resources on each UAV to the computing tasks waiting for offloading in the ground users according to the decisions made on UAV assignment in the DQN module.

Let $\boldsymbol{T}_{m,t} = \{T_{n,t} | x_n^m(t) = 1, T_{n,t} \in \boldsymbol{T}\}$ denote a set of computing tasks assigned to U_m in time slot t, q_0 denote the computation queue at U_m in time slot t, and τ_0 denote the time needed to complete all tasks in q_0. For a task $T_{n,t}$ in $\boldsymbol{T}_{m,t}$, let $\tau_{n,t}^{tr}$ denote its transmission delay and $\tau_{n,t}^{c}$ denote its computation delay. To perform bandwidth resource allocation, we introduce a virtual computation queue during the allocation process, and let q_0' and τ_0' denote the virtual queue and the time needed to complete all tasks in q_0', respectively. Moreover, we define the minimum service delay of task $T_{n,t}$ as the overall time taken to complete task $T_{n,t}$, i.e., the time from the beginning of the current time slot t to the instant $T_{n,t}$ is completed in the case that it is processed right after the tasks in q_0'. Thus, we have $\tau_{n,t} = \max\{\tau_0', \tau_{n,t}^{tr}\} + \tau_{n,t}^{c}$.

The main BRA process in the proposed algorithm is described as follows:

(1) Pre-allocate available bandwidth resources of U_m evenly to all the computing tasks in $\boldsymbol{T}_{m,t}$.
(2) Initialize the virtual computation queue q_0', and set $q_0' = q_0$ and $\tau_0' = \tau_0$.
(3) Compute the transmission delay $\tau_{n,t}^{tr}$ of each task in $\boldsymbol{T}_{m,t}$ based on the bandwidth resources allocated to the task. Compute the computation delay $\tau_{n,t}^{c}$ of each task in $\boldsymbol{T}_{m,t}$ according to Eq. (3).
(4) Compute the minimum service delays of those tasks in $\boldsymbol{T}_{m,t}$ but not in q_0', i.e., $\tau_{n,t}$.
(5) Select the task with the minimum $\tau_{n,t}$ from those tasks in $\boldsymbol{T}_{m,t}$ but not in q_0' and add the task to the end of q_0'. Then, update $\tau_0' = \min\{\tau_{n,t} | T_{n,t} \in \boldsymbol{T}_{m,t}\}$.
(6) Repeat (4)-(5) until all the tasks in $\boldsymbol{T}_{m,t}$ have been added into q_0'.
(7) For each task $T_{n,t}$ in q_0', compare its transmission delay (i.e., $\tau_{n,t}^{tr}$) with the service delay of the task right before $T_{n,t}$, denoted by $\tau_{n',t}$. If $\tau_{n,t}^{tr} < \tau_{n',t}$, the bandwidth resources allocated to $T_{n,t}$ are considered redundant, and $T_{n,t}$ is thus called a redundant task; otherwise, if $\tau_{n,t}^{tr} > \tau_{n',t}$, the bandwidth resources allocated to $T_{n,t}$ are considered insufficient, and $T_{n,t}$ is thus called an insufficient task.
(8) Estimate the amount of bandwidth resources required by each task in $\boldsymbol{T}_{m,t}$ using binary search. Reallocate the redundant resources of those redundant tasks to those insufficient tasks under constraints (12d) and (12e).
(9) Repeat (2)-(8) until that no redundant bandwidth resources can be allocated under constraints (12d) and (12e), there is no task with insufficient bandwidth resources, or the number of repetitions reaches a given threshold.

For learning the GNN module and the DQN module, the lower layer uses two groups of neural networks: evaluation networks and target networks. Both the GNN module and the DQN module are constructed by several evaluation networks and several target networks, respectively. The evaluation networks are used to choose actions while the target networks are used to assist the learning process of the evaluation networks. The target networks and the evaluation networks have the same structures and initial parameters. The parameters of the evaluation networks are updated by minimizing a loss function given by

$$\mathcal{L}_{\text{low}}(\theta) = \frac{1}{S}\sum_{S}\frac{1}{N}\left(y(t) - \sum_{n=1}^{N} Q(o_n(t), \{x_n^m(t)\}; \theta)\right)^2,$$
$$y(t) = r(t) + \gamma \sum_{n=1}^{N} \max_{\{x_n^m(t+1)\}} Q(o_n(t+1), \{x_n^m(t+1)\}; \theta'), \quad (19)$$

where $o_n(t)$ denotes the input of the DQN module for G_n in time slot t and $o_n(t) = \{\{s_m(t)\}, h_n''(t)\}, Q(o_n(t), \{x_n^m(t)\})$ denotes the action values output by the DNN in the DQN module, θ denotes the parameters of the evaluation networks, θ' denotes the parameters of the target networks, γ is a discount factor and S is the number of sampled experiences. The parameters of the target networks are updated according to a soft update policy:

$$\theta' \leftarrow \kappa\theta + (1-\kappa)\theta', \quad (20)$$

where κ is an update coefficient.

Algorithm 1 Lower-Layer Optimization Algorithm

```
Initialize network parameters θ, θ' and a replay buffer;
for episode = 1, ..., N_epi do
    Initialize the environment;
    for t = 1, ..., T do
        Obtain a state s_t;
        Input s_t into the GNN module and get extracted features h'';
        Input h'' into the DQN module and get action values;
        Choose an action on UAV assignment for each task;
        foreach U_m ∈ U do
            Pre-allocate available bandwidth resources evenly to all tasks assigned to U_m;
            while There is redundant bandwidth resource do
                Get the service order of all tasks;
                Reallocate bandwidth resources among tasks;
            end
        end
        Execute the joint action a_t;
        Obtain a reward r_t and a next state s_{t+1} ;
        Store (s_t, a_t, r_t, s_{t+1}) into the replay buffer;
    end
    Sample a batch of experiences from the replay buffer;
    Update θ according to (19);
    Update θ' according to (20).
end
```

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

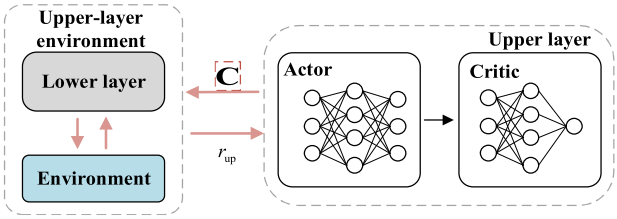


Fig. 4. Illustration of the upper layer.

The main procedure of the lower-layer optimization algorithm is described in Algorithm 1.

C. Upper-Layer Optimization Algorithm

The objective of the upper-layer optimization is to learn an optimal policy for making an optimal decision on service caching such that the average service delay of all computing tasks in the system is minimized. Given the computation offloading policy learnt by the lower layer, the upper-layer optimization problem can be formulated as

P3: objective:

$$\min_{\boldsymbol{C}} \quad D \tag{21a}$$

$$\text{subject to} \quad c_m^l \in \{0,1\}, \forall S_l \in \mathcal{L}, U_m \in \mathcal{U}, \tag{21b}$$

$$\sum_{l=1}^{L} c_m^l d_l \le C_m, \forall U_m \in \mathcal{U}, \tag{21c}$$

where constraints (21b)-(21c) are the same as constraints (11f)-(11g).

To solve problem **P3**, we also model the problem as an MDP. In the MDP, a virtual controller is introduced at the upper layer as an agent, which chooses actions on service caching. The environment of the upper layer is the combination of the lower-layer environment and the lower-layer policy. For the upper layer, the lower layer can be regarded as a black box. In other words, the upper layer can update its policy according to feedbacks from its environment no matter what specific policy the lower layer learns.

The essential elements of the modeled MDP for **P3** are defined as follows:

(1) *State:* The state of the upper-layer environment includes the positions of all UAVs in the system and the cache sizes of all service programs.

(2) *Action:* The action of the upper layer is to perform service caching for all UAVs in the system, i.e., to determine the service caching matrix $\boldsymbol{C}$. The service caching matrix is determined before each service period begins.

(3) *Reward:* The upper layer makes a decision on service caching before a service period and will obtain a reward from its environment after the service period. The reward of the upper layer is defined as an opposite value of the average service delay of all tasks in a service period, i.e.,

$$r_{\text{up}} = -D. \tag{22}$$

The upper layer uses an actor-critic structure and is composed of an actor network and a critic network, as illustrated in Fig. 4. The actor network is used to generate a service caching weight matrix, which evaluates the performance of caching different service programs on different UAVs. Based on the service caching weight matrix, the service caching matrix $\boldsymbol{C}$ can be generated by caching the service programs on each UAV in a descending order of the corresponding weights in the service caching matrix under constraint (21c). The critic network is used to evaluate the expected value of the reward r_{up}, which can be obtained based on the generated weight matrix.

In the learning process of the upper layer, the parameters of the actor network and critic network are updated to obtain an optimal service caching policy. Specifically, the parameters of the critic network are updated by minimizing the following loss function:

$$\mathcal{L}_{\text{up}}(\phi_Q) = \frac{1}{S}\sum_{S}(r_{\text{up}} - Q_{\text{up}}(s, \boldsymbol{\omega}; \phi_Q))^2, \tag{23}$$

where ϕ_Q denotes the parameters of the critic network, s denotes a state of the upper-layer environment, $\boldsymbol{\omega}$ denotes the service caching weight matrix generated by the actor network, $Q_{\text{up}}(s, \boldsymbol{\omega}; \phi_Q)$ represents the evaluated value given by the critic network, and S is the number of sampled service periods. Meanwhile, the parameters of the actor network are updated through policy gradient:

$$\nabla_{\phi_\mu}\mathcal{J} = \nabla_{\omega} Q_{\text{up}}(s, \boldsymbol{\omega}; \phi_Q)\nabla_{\phi_\mu}\mu(s; \phi_\mu), \tag{24}$$

where ϕ_μ denotes the parameters of the actor network, μ represents the policy of the actor network, according to which we have $\boldsymbol{\omega} = \mu(s; \phi_\mu)$.

Algorithm 2 Upper-Layer Optimization Algorithm

```
Initialize network parameters φ^μ, φ^Q;
for episode = 1, ..., N_epi do
    Obtain the state s including UAVs' positions and service
     program sizes;
    Input s into the actor network and generate a service
     weight matrix ω;
    Determine a service caching matrix C according to ω ;
    Execute the service caching action C;
    Run Algorithm 1 and obtain a reward r_up;
    Update φ_Q using (23);
    Update φ_μ using (24).
end
```

The main procedure of the upper-layer optimization algorithm is described in Algorithm 2.

D. Computational Complexity

For the upper layer, let N_{up} denote the number of neurons at each layer of the neural networks and L_{up} denote the number of layers in each neural network. The computational complexity of generating a service caching decision is $O(P_{up}) = O\left(\sum_{i=1}^{L_{up}} N_{up}^2\right)$.

For the lower layer, let N_{GNN} denote the feature dimension of a vertex in the GNN module, N_{DNN} denote the number of neurons at each layer of the DNN in the DQN module, L_{DNN} denote the number of layers in the DNN, and K denote the iteration threshold in the BRA module. Meanwhile, the GNN module has two attentional layers and Z denotes the number of attention heads on each attentional

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

TABLE I
KEY SIMULATION PARAMETERS

Parameter	Value	Parameter	Value
N	20	B	1 MHz
M	7	N_B	5
L	10	P_G	0.2 W
f_U	2 GHz	D_n	[0.5, 5] Mb
p_0	0.7	F_n	[0.1, 1] GHz
α	0.8	d_l	[1, 5] Mb
C_U	20 Mb	N_0	-130 dBm
Δt	1 s	$(\eta_{LoS},\eta_{NLoS},a,b)$	(1, 20, 9.61, 0.16)

layer. The computational complexity of the GNN module is $O(2Z(NN_{GNN}^2 + N^2 N_{GNN}))$. The computational complexity of the DQN module is $O(P_{DQN}) = O\left(\sum_{i=1}^{L_{DNN}} N_{DNN}^2\right)$. The computational complexity of the BRA module based on binary search is $O(KMN)$. Therefore, the computational complexity of generating a computation offloading decision is $O(2Z(NN_{GNN}^2 + N^2 N_{GNN}) + P_{DQN} + KMN)$.

V. PERFORMANCE EVALUATION

In this section, we present simulation results to evaluate the performance of the proposed DRL-based joint optimization algorithm.

A. Simulation Settings

In the simulation experiments, an MUeM system serves a circular urban area with a radius of 250 m, which consists of seven hexagons in a cellular pattern. At the center of each hexagon, there is one UAV hovering at a fixed altitude of 150m [24]. A number of ground users are randomly distributed in the area and move following a random waypoint mobility model [25] with a moving speed ranging from 1 to 3 m/s and a pause time ranging from 0 to 5 s. The other parameters used in the experiments are given in Table I. The values of $(\eta_{LoS}, \eta_{NLoS}, a, b)$ are set according to [22].

The simulation experiments were conducted using Python 3.8 and Pytorch 1.12.1. In the proposed learning framework, the actor network and critic network at the upper layer, and the DNN at the lower layer are fully connected two-layer neural networks with 128 neurons at each layer. The learning rate of the lower layer is 0.001. For the upper layer, the learning rates of the actor network and the critic network are set to 0.00001 and 0.0001, respectively. The discount factor (γ) is set to 0.8 and the batch size of sampled experiences (S) is set to 32.

In the evaluation, we consider the following three baselines:

(1) Popular caching and GNN-based offloading (PoG): a service program is cached on a UAV only according to its content popularity (i.e., the probability of the service program required by a computing task) under the cache capacity constraint. Meanwhile, UAV assignment and bandwidth resource allocation decisions are made by the lower-layer optimization algorithm.

(2) DNN-based caching and offloading (DNN): Service caching decisions are made by the upper-layer optimization algorithm, and UAV assignment and bandwidth resource allocation decisions are made by DNNs.

(3) Random caching and random offloading (Random): a service program is randomly cached on a UAV under the cache capacity constraint. Each computing task is randomly offloaded to a UAV which caches its required service program.

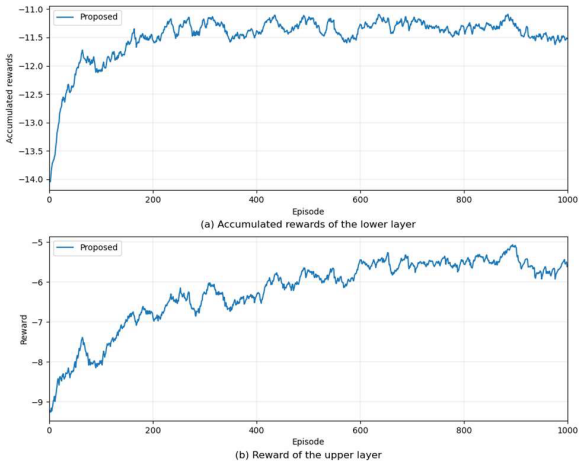


Fig. 5. The convergence performance of the proposed algorithm.

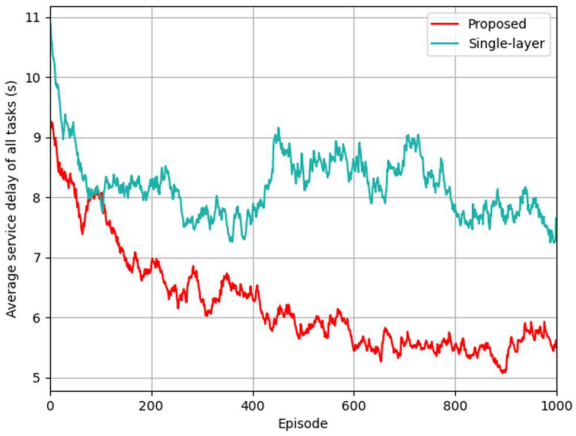


Fig. 6. Comparison of the average service delay convergence.

B. Simulation Results

Fig. 5 shows the convergence performance of the proposed algorithm. Fig. 5(a) shows the accumulated reward of the lower layer and Fig. 5(b) shows the reward of the upper layer. It is seen that both the accumulated reward of the lower layer and that of the upper layer initially increase quickly and gradually converge to stable values.

To evaluate the performance of the proposed two-layer learning framework, we implement a single-layer DRL baseline (Single-layer) that jointly makes service caching and computation offloading decisions. Fig. 6 shows the convergence performance in the average service delay with the proposed two-layer algorithm and the Single-layer. It is observed that Single-layer shows slower and less stable convergence compared with the proposed algorithm. This is because in the considered system service caching and computation offloading are performed at different time granularities.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

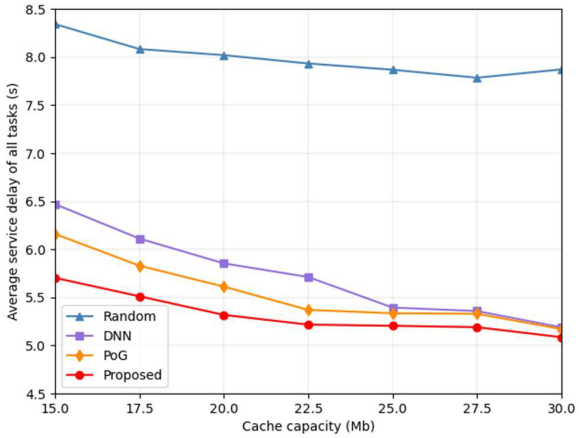


Fig. 7. The average service delay vs. the cache capacity of a UAV.

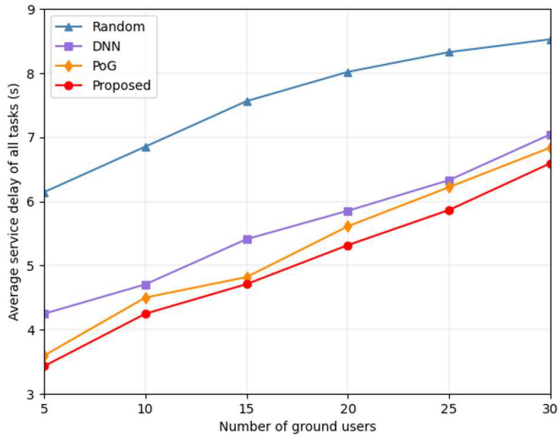


Fig. 8. The average service delay vs. the number of ground users.

Moreover, the service caching decision made before each service period determines the action space of the computation offloading in the service period. Using a single-layer neural network to jointly make service caching and computation offloading decisions increases the action space and thus leads to unsatisfactory convergence performance. In contrast, the proposed algorithm uses a two-layer learning framework to separate the time granularities in making service caching and computation offloading decisions, which results in better convergence performance.

Fig. 7 shows the average service delays of all computing tasks in the system vs the cache capacity of each UAV with the proposed algorithm, and the DNN, Random, and PoG algorithms, respectively. It is observed that as the cache capacity increases, the average service delays with all the four algorithms gradually decrease. The reason is that when the cache capacity of each UAV increases, each UAV can cache more types of service programs and has the ability to process more computing tasks requiring different types of service programs. Thus, when making an offloading decision on a computing task, there are more alternative UAVs and a smaller service delay can be obtained for the task. On the other hand, the average service delay with the proposed algorithm is smaller than those with the other three algorithms. Compared with Random and PoG, the proposed algorithm can make service caching decisions more efficiently for the reason that the service caching decisions in the proposed algorithm are made in a centralized manner and have considered global optimality. Compared with DNN, the lower layer used in the proposed algorithm has a stronger ability to extract information from input states and thus can make better offloading decisions with a smaller average service delay. Moreover, as the cache capacity increase beyond 25 Mb, the average service delays with DNN, PoG, and the proposed algorithm decrease more slowly. Meanwhile, the performance gap between DNN and the proposed algorithm and that between PoG and the proposed algorithm become smaller. This is because in this case, each UAV is able to cache most service programs and different service caching policies have little effect on the simulation results.

Fig. 8 shows the average service delays vs the number of ground users in the system with the four algorithms, respectively. It is seen that as the number of ground users increases, the average service delays with all the four algorithms increase as well. This is because when the number of ground users increases, the number of computing tasks generated by the ground users increases as well. Each computing task is allocated less bandwidth resources and needs to wait for a longer time in the computation queue at a UAV before being processed. On the other hand, the average service delay with the proposed algorithm is smaller than those with the other three algorithms.

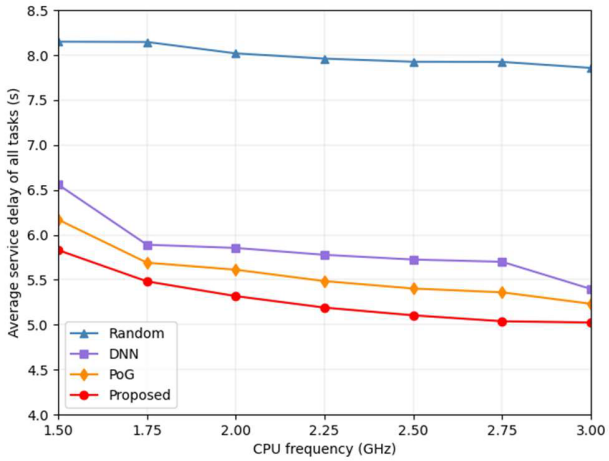


Fig. 9. The average service delay vs. the CPU frequency.

Fig. 9 shows the average service delays vs the CPU frequency of each UAV with the four algorithms, respectively. It is observed that as the CPU frequency increases, the average delays with all four algorithms decrease. The reason is that as the CPU frequency increases, each computing task can be processed more quickly. On the other hand, the average service delay with the proposed algorithm is smaller than those with the other three algorithms.

Fig. 10 shows the average service delays vs the bandwidth of a subchannel with the four algorithms, respectively. It is observed that as the bandwidth of a subchannel increases, the average service delays with all the four algorithms decrease gradually and tend to be stable. The reason is that according to Shannon's theory the transmission rate of a ground user increases with the bandwidth allocated to the user increasing.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

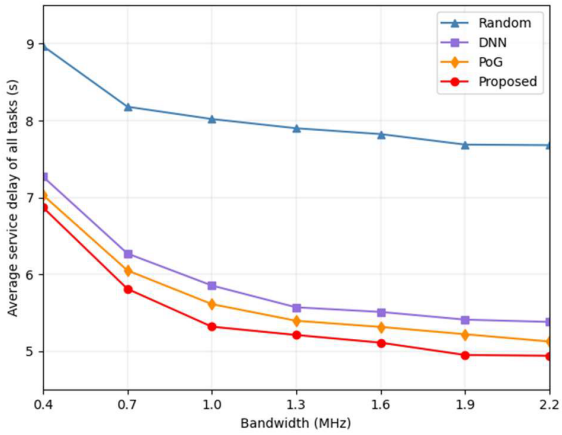


Fig. 10. The average service delay vs the bandwidth of a subchannel.

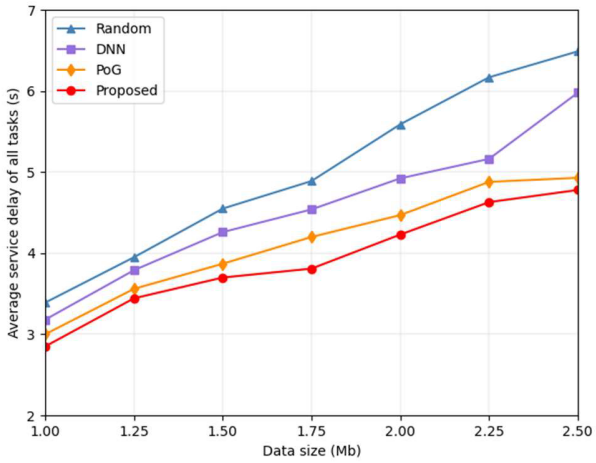


Fig. 11. The average service delay vs the data size of a computing task.

However, the transmission rate of a channel is limited. As the bandwidth allocated to a ground user increases, the transmission rate of the user will gradually approach a limit (i.e., Shannon Limit). Thus, the average transmission delay will tend to be stable and accordingly the average service delay.

Fig. 11 shows the average service delays vs the data size of a computing task (D_n) with the four algorithms, respectively. It is observed that as the data size of each computing task increases, the average service delays with all the four algorithms gradually increase. The reason is that when the data size of a task increases, it will take more time to transmit the task. Meanwhile, a task may need to wait a longer time before being allocated bandwidth resources.

Fig. 12 shows the average service delays vs the transmission power of a ground user with the four algorithms, respectively. It is observed that as the transmission power of a ground user increases, the average delays with all the four algorithms gradually decrease. The reason is that when the transmission power of a ground user increases, the transmission rate of the user will increase and a task can be transmitted to a UAV faster. On the other hand, the average service delay with the proposed algorithm is smaller than those with the other three algorithms.

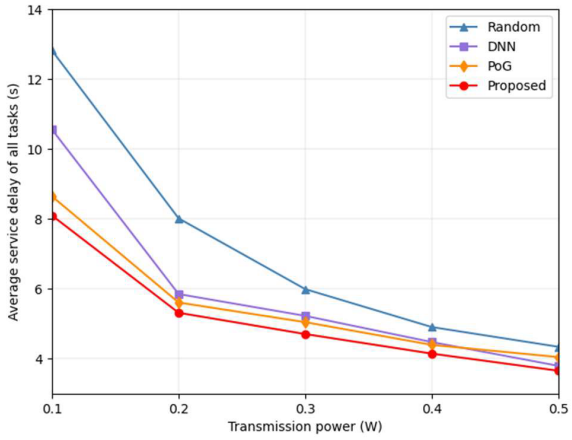


Fig. 12. The average service delay vs the transmission power of a ground user.

VI. Conclusion

In this paper, we investigated the joint service caching and computation offloading optimization problem in an MUeM system and formulated the problem as an NLIP problem. To solve the formulated problem, we first decomposed the problem into two subproblems and then introduced a two-layer learning framework to solve the subproblems. The upper layer is used to learn an optimal policy for making an optimal decision on service caching and the lower layer is used to learn an optimal policy for making an optimal decision on computation offloading. Based on the learning framework, we further proposed a DRL-based joint optimization algorithm to perform service caching and computation offloading, which consists of a lower-layer algorithm and an upper-layer algorithm. The lower-layer algorithm uses a GNN and an attention mechanism to extract relevance information on the states of the ground users in the system and uses a DQN to make UAV assignment decisions based on the extracted information. The upper-layer algorithm uses an actor-critic structure to make optimal service caching decisions. The simulation results demonstrate that the proposed algorithm can significantly reduce the average service delay of all computing tasks in the system compared with the three benchmark algorithms. However, the objective of this work is to minimize the average service delay of all computing tasks generated by ground users in the system, which may not be able to guarantee delay fairness among individual computing tasks. As a result, tasks with unfavorable channel conditions or limited resources may experience relatively larger delays. In future work, we will take into account delay fairness in the problem formulation in order to address this issue.

References

[1] M. Vaezi et al., "Cellular, wide-area, and non-terrestrial IoT: A survey on 5G advances and the road toward 6G," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 2, pp. 1117–1174, 2nd Quart., 2022.

[2] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1628–1656, 3rd Quart., 2017.

[3] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, Feb. 2018.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.

[4] X. Xia, S. M. M. Fattah, and M. A. Babar, "A survey on UAV-enabled edge computing: Resource management perspective," *ACM Comput. Surveys*, vol. 56, no. 3, pp. 1–36, Mar. 2024.
[5] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2334–2360, 3rd Quart., 2019.
[6] Y. Luo, Y. Wang, Y. Lei, C. Wang, D. Zhang, and W. Ding, "Decentralized user allocation and dynamic service for multi-UAV-enabled MEC system," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 1306–1321, Jan. 2024.
[7] W. Liu, B. Li, W. Xie, Y. Dai, and Z. Fei, "Energy efficient computation offloading in aerial edge networks with multi-agent cooperation," *IEEE Trans. Wireless Commun.*, vol. 22, no. 9, pp. 5725–5739, Sep. 2023.
[8] L. X. Nguyen, Y. K. Tun, T. N. Dang, Y. M. Park, Z. Han, and C. S. Hong, "Dependency tasks offloading and communication resource allocation in collaborative UAV networks: A Metaheuristic approach," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 9062–9076, May 2023.
[9] F. Pervez, A. Sultana, C. Yang, and L. Zhao, "Energy and latency efficient joint communication and computation optimization in a multi-UAV-assisted MEC network," *IEEE Trans. Wireless Commun.*, vol. 23, no. 3, pp. 1728–1741, Mar. 2024.
[10] H. Yu, S. Leng, and F. Wu, "Joint cooperative computation offloading and trajectory optimization in heterogeneous UAV-swarm-enabled aerial edge computing networks," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17700–17711, May 2024.
[11] Z. Kuang, H. Wang, J. Li, and F. Hou, "Utility-aware UAV deployment and task offloading in multi-UAV edge computing networks," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14755–14770, Apr. 2024.
[12] Z. Qin et al., "AoI-aware scheduling for air-ground collaborative mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 2989–3005, May 2023.
[13] S. Bi, L. Huang, and Y.-J.-A. Zhang, "Joint optimization of service caching placement and computation offloading in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4947–4963, Jul. 2020.
[14] G. Zhang, S. Zhang, W. Zhang, Z. Shen, and L. Wang, "Joint service caching, computation offloading and resource allocation in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 5288–5300, Aug. 2021.
[15] D. Wang, Y. Bai, G. Huang, B. Song, and F. R. Yu, "Cache-aided MEC for IoT: Resource allocation using deep graph reinforcement learning," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11486–11496, Jul. 2023.
[16] C. Shang, Y. Huang, Y. Sun, and M. Guizani, "Joint computation offloading and service caching in mobile edge-cloud computing via deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 24, pp. 40331–40344, Dec. 2024.
[17] G. Zheng, C. Xu, M. Wen, and X. Zhao, "Service caching based aerial cooperative computing and resource allocation in multi-UAV enabled MEC systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10934–10947, Oct. 2022.
[18] H. Yan, H. Li, X. Xu, and M. Bilal, "UAV-enhanced service caching for IoT systems in extreme environments," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 26741–26750, Aug. 2024.
[19] Y. Zhao et al., "Joint content caching, service placement, and task offloading in UAV-enabled mobile edge computing networks," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 1, pp. 51–63, Jan. 2024.
[20] C. Li, K. Jiang, Z. Zhang, C. Xiong, and S. Wan, "Joint service caching and computation offloading scheme with 3D UAV deployment for ICVs in UAV-assisted VEC," *IEEE Trans. Commun.*, vol. 73, no. 11, pp. 10886–10899, Nov. 2025.
[21] L. Breslau, P. Cao, F. Li, G. M. Phillips, and S. Shenker, "Web caching and Zipf-like distributions: Evidence and implications," *IEEE INFOCOM*, Mar. 1999, pp. 126–134.
[22] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014.
[23] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
[24] V. H. M. Donald, "Advanced mobile phone service: The cellular concept," *Bell Syst. Tech. J.*, vol. 58, no. 1, pp. 15–41, Jan. 1979.
[25] C. Bettstetter, G. Resta, and P. Santi, "The node distribution of the random waypoint mobility model for wireless ad hoc networks," *IEEE Trans. Mobile Comput.*, vol. 2, no. 3, pp. 257–269, Jul. 2003.



Luyinru Yang received the bachelor's degree in information engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2019. She is currently pursuing the Ph.D. degree with the School of Information Science and Engineering, Southeast University, Nanjing. Her research interests include edge computing, unmanned aerial vehicles, and reinforcement learning.



Jun Zheng (Senior Member, IEEE) received the Ph.D. degree in electrical and electronic engineering from The University of Hong Kong, Hong Kong, in 2000. He is currently a Full Professor with the School of Information Science and Engineering, Southeast University (SEU), China, and a member with Purple Mountain Laboratories, China. Before joining SEU in 2008, he was with the School of Information Technology and Engineering, University of Ottawa, Canada. He has co-authored (first author) two books published by Wiley-IEEE Press, and has authored or co-authored over 200 technical papers in refereed journals and peer-reviewed conference proceedings. His current research interests include mobile communication networks, vehicular ad hoc networks, and UAV-connected networks, focused on network architectures and protocols. He is a Senior Member of the IEEE Communications Society and the IEEE Vehicular Technology Society. He has also served as a TPC member for a number of international conferences and symposiums. He was a recipient of the 2021 IEEE ComSoc Communications Software Technical Committee "Technical Achievement Award" and the 2023 IEEE ComSoc Communications Switching and Routing Technical Committee "Distinguished Technical Achievement Award." He was a co-recipient of the Best Paper Awards at IEEE ICC 2014 and WCSP 2018. He was the Chair of the IEEE ComSoc Communications Switching and Routing Technical Committee. He has served as the Founding General Chair for AdHocNets 2009; the General Chair for AccessNets 2007, AdHocNets 2018/2019, and ICNC 2024; and the TPC or Symposium Co-Chair for a number of international conferences and symposia, including IEEE ICC 2009/2011/2015/2021 and GLOBECOM 2008/2010/2012/2018/2019. He serves as the Chair for the IEEE Vehicular Technology Society Nanjing Chapter. He is an Editorial Board Member of several international journals, including IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY and IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING. He has co-edited 14 special issues for different refereed journals and magazines, including *IEEE Communications Magazine*, *IEEE Network*, and IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, all as the Lead Guest Editor.



Yuan Zhang received the bachelor's and Ph.D. degrees from Southeast University, Nanjing, China, in 1998 and 2004, respectively. Since 2005, he has been a Faculty Member with the National Mobile Communications Research Laboratory, Southeast University. His research interests include wireless communications, networking, and computing. He has published more than 50 technical papers in journals and conference proceedings. He was a co-recipient of two Best Paper Awards at ICC 2014 and ICCC 2019.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:33:32 UTC from IEEE Xplore. Restrictions apply.